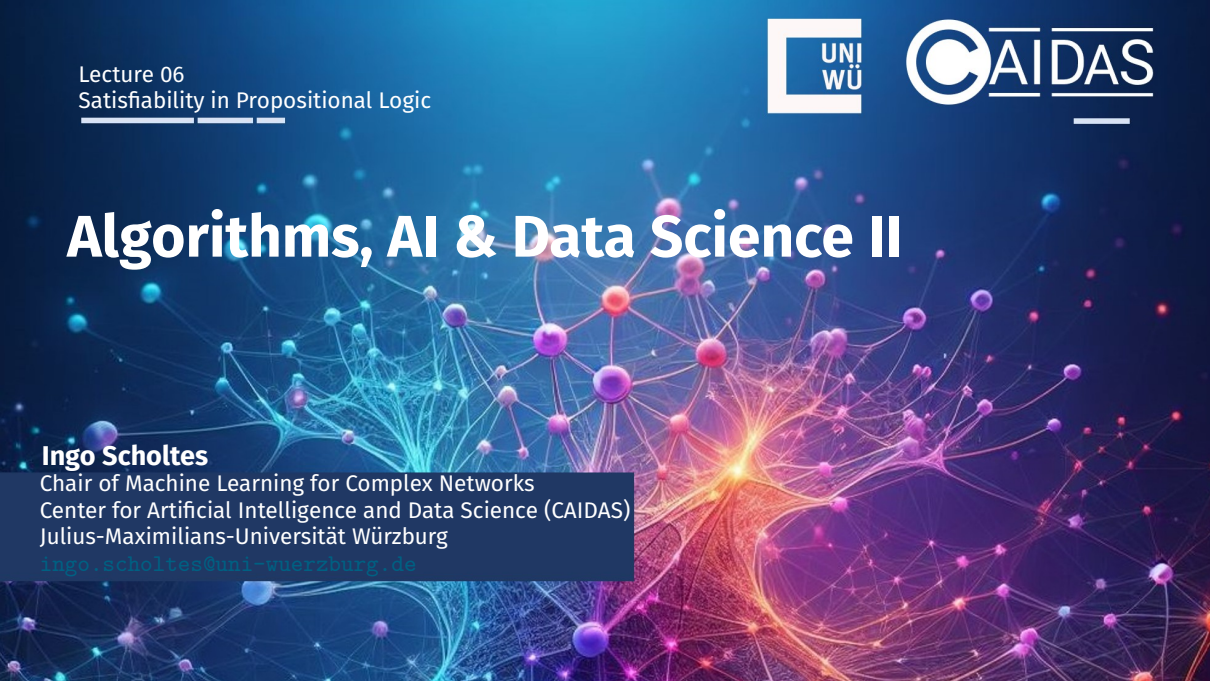
Lecture 06
Satisfiability in Propositional Logic

# Algorithms, AI & Data Science II



**Ingo Scholtes**
Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

## Notes:

- **L06: Satisfiability in Propositional Logic** 19.05.2025
- **Educational objective:** We define the satisfiability problem in propositional logic and consider the DPLL algorithm. We introduce logical inference and show how we can use rules of inference to infer logical conclusions.
  - Satisfiability in Propositional Logic
  - Logical Inference and Rules of Inference
  - Forward Reasoning
- **Handout version:** *2025/05/19 09:58:37*

# Motivation

- we introduced **propositional logic**, a context-free formal language that can be used to **represent knowledge**
- propositional logic consists of **logically connected propositions**, e.g.

$$X_0 \wedge X_1 \implies X_2$$

- introducing Boolean algebra, we took an **algebraic perspective on formal logics**

| $X_0$ | $X_1$ | $X_0 \implies X_1$ | $X_0 \wedge \neg X_1$ |
|---|---|---|---|
| False | False | True | False |
| False | True | True | False |
| True | False | False | True |
| True | True | True | False |

**open questions**

- how can we use propositional logic to **automatically reason** about the world?
- how can we check whether a **formula can possibly evaluate to true** (for any variable assignment)?

**Notes:**

# Satisfiability problem

- for propositional formula $F$ with $n$ variables $X_i$ we call $\mathcal{A} : (X_0, \ldots, X_n) \to \{True, False\}^n$ **assignment**
- $\mathcal{A}(F)$ denotes truth value of $F$ for assignment $\mathcal{A}$
- we say that **assignment $\mathcal{A}$ satisfies $F$** iff $\mathcal{A}(F) = True$ (we write $\mathcal{A} \models F$ and call $\mathcal{A}$ "model" for $F$)

**satisfiability for propositional logic**

Propositional formula $F$ is **satisfiable** iff there is an assignment $\mathcal{A}$ with $\mathcal{A} \models F$. Otherwise we say that $F$ is **unsatisfiable**.

- how can we solve **word decision problem** for language
  $L = \{F | F \text{ is satisfiable formula in propositional logic}\}$
- we can use **truth table enumeration algorithm**, i.e. for propositional logic satisfiability is **decidable**

**example 1**

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X_0} \vee X_1) \wedge (\bar{X_0} \vee \bar{X_1}) \wedge (X_0 \vee \bar{X_2})$$

For $\mathcal{A}(X_0, X_1, X_2) = (False, True, False)$ we have $\mathcal{A}(\mathcal{F}) = True$

$\mathcal{A} \models F$ and thus $F$ is **satisfiable**

**example 2**

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X_0} \vee X_1) \wedge (\bar{X_0} \vee \bar{X_1}) \wedge (X_0 \vee \bar{X_1})$$

$\mathcal{A}(F) = False$ for **any assignment $\mathcal{A}$** and thus $F$ is **not satisfiable**

**Notes:**

# Cook-Levin theorem

- for formula with $n$ variables, **truth table has $2^n$ rows**, i.e. time complexity is in $2^{O(n)}$
- can we find a **faster (polynomial-time) algorithm**?

**Cook-Levin theorem**

**Satisfiability problem in propositional logic is NP-complete**, i.e. there is no known polynomial-time algorithm that can decide whether a given formula is satisfiable or not.

- central result in computer science because **many problems can be reduced to satisfiability**
- reducing a problem $A$ to (Boolean) satisfiability implies that **satisfiability** is at least as difficult as $A$
- any polynomial-time algorithm for satisfiability would imply that we can also solve $A$ in polynomial time

Stephen Arthur Cook, born 1939

image credit: Wikimedia Commons, Jiří Janíček, CC-BY-SA

**Notes:**

# SAT-solvers and backtracking

- consider satisfiability problem for propositional formula $F$ in **conjunctive normal form**
- confirming that $F$ is unsatisfiable may require exponential time but clever **search algorithms** can **find satisfying assignment** (iff $F$ is satisfiable)
- we can use trial-and-error based **backtracking algorithm** to find $\mathcal{A}$ with $\mathcal{A} \models F$
  - select variable and **assign truth value**
  - use **annulment and identity law** to simplify formula
  - return **satisfiable if simplified formula is empty**
  - abandon assignment ("backtrack") if simplified formula contains empty clause (i.e. is unsatisfiable)
- SAT-solvers are important basis for **automated theorem proving and logical inference**

**backtracking algorithm**

1. **choose variable** $X_i$ and set $X_i = \mathit{True}$ or $X_i = \mathit{False}$
2. **simplify** $F$ by
   (i) removing clauses where literal evaluates to $\mathit{True}$
   (ii) removing literals that evaluate to $\mathit{False}$ from remaining clauses
3. if **simplified formula is empty** then $F$ is satisfiable, i.e. return True
4. if **simplified formula contains empty clause**, assign complementary value to $X_i$ and repeat (or return False if we have tried all assignments)

**example 1**

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X}_0 \vee X_1) \wedge (\bar{X}_0 \vee \bar{X}_1) \wedge (X_0 \vee \bar{X}_2)$$

setting $X_0 = \mathit{True}$ **we can simplify** to

$$F = \cancel{(X_0 \vee X_1)} \wedge (X_1 \vee X_2) \wedge (\cancel{\bar{X}_0} \vee X_1) \wedge (\cancel{\bar{X}_0} \vee \bar{X}_1) \wedge \cancel{(X_0 \vee \bar{X}_2)}$$

**Notes:**

# Group exercise 06-01



Apply backtracking to check the satisfiability of the following formula

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X}_0 \vee X_1) \wedge (\bar{X}_0 \vee \bar{X}_1) \wedge (X_0 \vee \bar{X}_2)$$

1. we set $X_0 = \mathit{True}$ and simplify to

$$F = \cancel{(X_0 \vee X_1)} \wedge (X_1 \vee X_2) \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge \cancel{(X_0 \vee \bar{X}_2)}$$

2. we set $X_1 = \mathit{True}$ and simplify to

$$F = \cancel{(X_0 \vee X_1)} \wedge \cancel{(X_1 \vee X_2)} \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \underbrace{\cancel{(\bar{X}_0 \vee \bar{X}_1)}}_{\text{empty clause!}} \wedge \cancel{(X_0 \vee \bar{X}_2)}$$

3. we backtrack, set $X_1 = \mathit{False}$ and simplify to

$$F = \cancel{(X_0 \vee X_1)} \wedge \cancel{(X_1 \vee X_2)} \wedge \underbrace{\cancel{(\bar{X}_0 \vee X_1)}}_{\text{empty clause}} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge \cancel{(X_0 \vee \bar{X}_2)}$$

**Notes:**

# Group exercise 06-01



1. we backtrack, set $X_0 = \mathit{False}$ and simplify to
$$F = (\cancel{X_0} \vee X_1) \wedge (X_1 \vee X_2) \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge (\cancel{X_0} \vee \bar{X}_2)$$

2. we set $X_1 = \mathit{True}$ and simplify to
$$F = \cancel{(X_0 \vee X_1)} \wedge \cancel{(X_1 \vee X_2)} \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge (\cancel{X_0} \vee \bar{X}_2)$$

3. we set $X_2 = \mathit{True}$ and simplify to
$$F = \cancel{(X_0 \vee X_1)} \wedge \cancel{(X_1 \vee X_2)} \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge \underbrace{(\cancel{X_0} \vee \cancel{\bar{X}_2})}_{\text{empty clause!}}$$

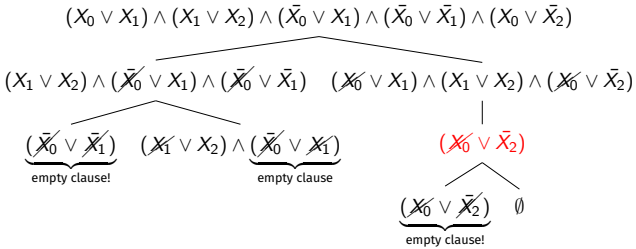
4. we backtrack, set $X_2 = \mathit{False}$ and simplify to
$$F = \cancel{(X_0 \vee X_1)} \wedge \cancel{(X_1 \vee X_2)} \wedge \cancel{(\bar{X}_0 \vee X_1)} \wedge \cancel{(\bar{X}_0 \vee \bar{X}_1)} \wedge \cancel{(X_0 \vee \bar{X}_2)}$$

Since the simplified formula does not contain any clauses (i.e. it is empty) the **original formula $F$ is satisfiable** for assignment $\mathcal{A} = (X_0, X_1, X_2) = (\mathit{False}, \mathit{True}, \mathit{False})$

**Notes:**

# Improving naive backtracking

- execution of backtracking algorithm defines a **search tree**
- cancel search along branch if **partial solution** cannot yield desired result
- naive backtracking is an **uninformed search strategy** that randomly assigns variables in each step
- idea: improve performance by using **smarter search strategy** that considers properties of propositional logic?
- is it reasonable to set $X_2 = \mathit{True}$ if formula contains **unit clause** $\bar{X}_2$?




formula is satisfiable for assignment

$\mathcal{A} = (X_0, X_1, X_2) = (\mathit{False}, \mathit{True}, \mathit{False})$

**Notes:**

# DPLL algorithm

- consider formula $F$ in CNF as a **set of clauses** $\{c_1, \ldots, c_k\}$
- **Davis-Putnam-Logemann-Loveland (DPLL) algorithm** improves basic backtracking with **two additional rules that speed up search**
  → M Davis, H Putnam, 1960
  → M Davis, G Logemann, D Loveland, 1961

**DPLL search rules**

1. **unit propagation**: if clause has single variable $X_i$ (i.e. it is "unit clause") assign value to $X_i$ such that clause evaluates to true, remove unit clause, and remove $X_i$ from clauses where it appears as complement
2. **pure variable elimination:** if variable $X_i$ has the same polarity in all clauses where it appears (i.e. it is "pure"), assign value to $X_i$ such that those clauses evaluate to *True* and remove those clauses

**DPLL algorithm**

**procedure** DPLL($F$)
  **while** there is unit clause $\{X_i\}$ in $F$ **do**
    $F \leftarrow$ unit-propagate($X_i$, $F$)
  **end while**
  **while** there is pure variable $\{X_i\}$ in $F$ **do**
    $F \leftarrow$ pure-variable-assign($X_i$, $F$)
  **end while**
  **if** $F$ is empty **then**
    return `True`
  **end if**
  **if** $F$ contains empty clause **then**
    **return** `False`
  **end if**
  $X_i \leftarrow$ choose-variable($F$)
  **return** DPLL$(F \wedge \{X_i\}) \vee$ DPLL$(F \wedge \{\bar{X}_i\})$
**end procedure**

**Notes:**

1. We use unit propagation because if a variable appears in a unit clause, the formula can only be satisfied with the assignment of a corresponding truth value
2. We use pure variable elimination because by setting the variable to a truth value that makes all clauses true in which it appears, those clauses do not impose constraints for the choice of other variables in the search tree, i.e. we can always satisfy all of those clauses just by setting the single pure variable accordingly. In other words, setting the pure variable to the complementary value can only make the search for a satisfying assignment harder (as we impose additional constraints for the other variables).

# Group exercise 06-02



- Use the DPLL algorithm to **find a satisfying assignment** for the following propositional formula $F$ in CNF.

$$(X_0 \vee X_1) \wedge \bar{X_1} \wedge (\bar{X_0} \vee X_2)$$

1. we first find the **unit clause** $\bar{X_1}$, i.e. we must necessarily assign $X_1 \leftarrow \mathit{False}$. We remove the unit clause $\bar{X_1}$ and remove $X_1$ from the first clause:

$$X_0 \wedge (\bar{X_0} \vee X_2)$$

2. In the simplified formula, there is another **unit clause** $X_0$, i.e. we must assign $X_0 \leftarrow \mathit{True}$. We remove the unit clause $X_0$ and remove $\bar{X_0}$ from the second clause.

$$X_2$$

3. We are left with a last unit clause, i.e. we must assign $X_2 \leftarrow \mathit{True}$. Since the resulting simplified formula is empty, $F$ **is satisfiable**.
4. We found (the only) satisfying assignment $\mathcal{A}(X_2, X_1, X_0) = (\mathit{True}, \mathit{False}, \mathit{True})$.

**Notes:**

# Group exercise 06-02



- Use the DPLL algorithm to show that the following propositional formula $F$ is **unsatisfiable**.

$$(X_1 \vee X_0) \wedge (X_0 \vee X_2) \wedge (\bar{X}_0 \vee X_1) \wedge (\bar{X}_2 \vee X_0) \wedge (\bar{X}_2 \vee \bar{X}_0) \wedge (X_2 \vee \bar{X}_0)$$

1. We first note that $X_1$ **is a pure variable** that always occurs with same polarity. Assigning $X_1 \leftarrow \mathit{True}$ makes all clauses where it appears $\mathit{True}$, i.e. we can remove those clauses.

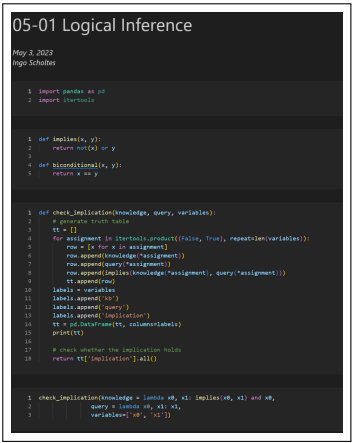
$$(X_0 \vee X_2) \wedge (\bar{X}_2 \vee X_0) \wedge (\bar{X}_2 \vee \bar{X}_0) \wedge (X_2 \vee \bar{X}_0)$$

2. We neither have pure variables nor unit clauses, so we must **choose a variable**, e.g. $X_0$. With $X_0 \leftarrow \mathit{True}$ we can simplify to $\bar{X}_2 \wedge X_2$
3. We choose the **unit clause** $X_2$, assign $X_2 \leftarrow \mathit{True}$ and remove $X_2$ from the second clause.
4. The simplified formula (only) contains an **empty clause**, i.e. we **backtrack and choose** $X_0 \leftarrow \mathit{False}$. We now simplify to the same formula as before: $\bar{X}_2 \wedge X_2$
5. Choosing the **unit clause** $X_2$ or $\bar{X}_2$ yields again the **empty clause**. We cannot backtrack anymore and conclude that the **original formula is unsatisfiable**.

**Notes:**

# Practice Session

- we implement the **DPLL algorithm** to check propositional satisfiability in `python`
- we use our implementation to check the satisfiability of propositional formulas.




**practice session**

see notebooks 06-01 in `gitlab` repository at

→ `https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2`

**Notes:**

# In summary …

- we explored the **satisfiability problem** in propositional logic
- propositional satisfiability is a **decidable but NP-complete problem**
- we considered backtracking, a naive uninformed search algorithm that can be used as a basis for **SAT-solvers**
- we considered the **backtracking-based DPLL algorithm** that uses unit-propagation and pure-variable elimination to speed up search
- SAT-solvers are important foundation for **logical inference**

DALL-E image generated for prompt “an image that represents the relation between formal logics and symbolic artificial intelligence”

image credit: www.bing.com

**Notes:**

# Self-study questions

1. Use truth table enumeration to determine the satisfiability of the following formula:

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X_0} \vee X_1) \wedge (\bar{X_0} \vee \bar{X_1}) \wedge (X_0 \vee \bar{X_2})$$

2. Use the DPLL algorithm to show that the following formula is not satisfiable:

$$F = (X_0 \vee X_1) \wedge (X_1 \vee X_2) \wedge (\bar{X_0} \vee X_1) \wedge (\bar{X_0} \vee \bar{X_1}) \wedge (X_0 \vee \bar{X_1})$$

3. Explain why empty clauses in the DPLL algorithm indicates that $F$ is not satisfiable.
4. Explain why an empty formula in the DPLL algorithm indicates that $F$ is satisfiable.
5. Give an example for an unsatisfiable formula without pure variables or unit clauses.
6. Explain the reasoning behind the unit propagation and the pure variable elimination rules in the DPLL algorithm.
7. Try to construct an example for a propositional formula in which the unit propagation and pure variable elimination rules in DPLL can not be applied a single time.
8. Investigate algorithms to solve the so-called 2-satisfiability problem (2-SAT) for propositional formulas in CNF with exactly two propositional variables. How does the complexity of this problem differ from the general satisfiability problem in propositional logic?

**Notes:**

# References

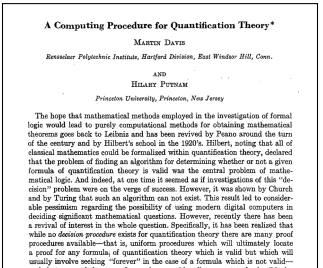
## reading list


- M Davis and H Putnam: **A Computing Procedure for Quantification Theory**. Journal of the ACM. 7 (3), 1960
- M Davis, G Logemann, D Loveland: **A Machine Program for Theorem Proving**, Communications of the ACM. 5 (7), 1961
- B Nebel: **Logics for Knowledge Representation**, Int. Encyc. Social and Behavioral Sciences, 2001
- S Hedman: **A First Course in Logic**, Oxford Texts in Logic, 2004
- U Schöning: **Logik für Informatiker**, Spektrum Akad. Verlag, 2005
- DE Knuth: **The Art of Computer Programming 4A: Combinatorial Algorithms**, chapter 7
- S Russel and P Norvig: **Artificial Intelligence: A modern Approach**, 2022, chapter 7


## acknowledgments


examples partly based on the following books and lectures:

- V Goranko: **First-Order Logic: Satisfiability, validity, logical consequence**, DTU, 2010
- C Meinel, M Mundhenk: **Mathematische Grundlagen der Informatik**, Springer, 2014
- S Russel and P Norvig: **Artificial Intelligence: A modern Approach**, 2022 → section 7.5

**A Computing Procedure for Quantification Theory***


MARTIN DAVIS

*Rensselaer Polytechnic Institute, Hartford Division, East Windsor Hill, Conn.*

AND

HILARY PUTNAM

*Princeton University, Princeton, New Jersey*


The hope that mathematical methods employed in the investigation of formal logic would lead to purely computational methods for obtaining mathematical theorems goes back to Leibniz and has been revived by Peano around the turn of the century and by Hilbert's school in the 1920's. Hilbert, noting that all of classical mathematics could be formalized within quantification theory, declared that the problem of finding an algorithm for determining whether or not a given formula of quantification theory is valid was the central problem of mathematical logic. And indeed, at one time it seemed as if investigations of this "decision" problem were on the verge of success. However, it was shown by Church and by Turing that such an algorithm can not exist. This result led to considerable pessimism regarding the possibility of using modern digital computers in deciding significant mathematical questions. However, recently there has been a revival of interest in the whole question. Specifically, it has been realized that while no decision procedure exists for quantification theory there are many proof procedures available—that is, uniform procedures which will ultimately locate a proof for any formula, of quantification theory which is valid but which will usually involve seeking "forever" in the case of a formula which is not valid—and that some of these proof procedures could well turn out to be feasible for use with modern computing machinery.

Hao Wang [9] and P. C. Gilmore [3] have each produced working programs which employ proof procedures in quantification theory. Gilmore's program employs a form of a basic theorem of mathematical logic due to Herbrand, and Wang's makes use of a formulation of quantification theory related to those studied by Gentzen. However, both programs encounter decisive difficulties with any but the simplest formulas of quantification theory, in connection with methods of doing propositional calculus. Wang's program, because of its use of Gentzen-like methods, involves exponentiation on the total number of truth-functional connectives, whereas Gilmore's program, using normal forms, involves exponentiation on the number of clauses present. Both methods are superior in many cases to truth table methods which involve exponentiation on the


* Received September, 1959. This research was supported by the United States Air Force through the Air Force Office of Scientific Research of the Air Research and Development Command, under Contract No. AF 49(638)-527. Reproduction in whole or in part is permitted for any purpose of the United States Government.

**Notes:**